

Département mathématiques et informatique  
UFR des Sciences

**Programmation Fonctionnelle (YSINL4B1)**  
**1 heure**

*Tout document (électronique ou papier) est interdit. L'accès à internet, les ordinateurs portables, téléphones portables, montres, clés usb et plus généralement tout matériel électronique sont interdits.*

**Consignes :**

- Vous devez ajouter votre numéro de place dans la salle sur ce sujet et le rendre à la fin de l'épreuve.
- Mettez vos réponses dans les espaces prévus sur le sujet.
- Le sujet avec vos réponses est à mettre à l'intérieur de la feuille de l'université qui comportera votre nom et prénom cachés.
- Toutes les fonctions définies pour les questions **doivent être signées**.

|                   |  |
|-------------------|--|
| Numéro étudiant : |  |
| Numéro de place : |  |

## 1 Évaluation et analyse des expressions. (5 pts)

Évaluer les expressions suivantes en expliquant en quelques mots le résultat trouvé.

1. `[ (\x -> even x) e | e <- [1..5] ]` (1pt)

Réponse :

2. `(snd . last) [(1,"Orange"), (2,"Banane"), (3,"Figue")]` (1pt)

Réponse :

3. `foldl (\x y -> x + y) 0 [1,2,3] == foldr (\y x -> x+y) 0 [1,2,3]` (1,5pts)

Réponse :

4. `map (*) [1,2,3,4]` (1,5pts)

Réponse :

## 2 Types et signatures des fonctions (3pts)

Donner, en les justifiant, les signatures polymorphes des fonctions suivantes : (on choisira le parenthésage minimal pour les signatures)

1. `const x y = x` (1pt)

Réponse :

2. `subst f g x = f x (g x)` (1pt)

Réponse :

3. `fix f x = f (fix f) x` (1pt)

Réponse :

## 3 Décomposition en facteurs premiers des entiers naturels. (7 pts)

Tout entier est représenté par une liste de couples (**facteur**, **exposant**) ordonnée selon les facteurs. Par exemple le nombre naturel 200 est représenté par  $2^3 \times 5^2$ . On définit les types suivants :

```
type Facteur = Int
type Exposant = Int
type Couple = (Facteur, Exposant)
type Representation = [Couple]
```

1. À l'aide d'une ZF-expression et la fonction `product` d'Haskell, définir la fonction (`eval xs`) qui évalue un entier ainsi représenté : (2 pts)

```
eval [] ==> 1
eval [(2,3), (5,2)] ==> 200
eval [(2,1), (5,1), (7,3)] ==> 3430
```

Réponse :

2. définir la fonction (`estPremier xs`) qui détermine si un entier (ainsi représenté) est premier : **(2 pts)**

```
estPremier [(2,3), (5,2)] ==> False
estPremier [(2,1)]      ==> True
```

Réponse :

3. définir la fonction (`pgcd xs ys`) qui calcule le pgcd (plus grand commun diviseur) de 2 entiers. **(3 pts)**

```
pgcd [(2,3), (5,2)] [(2,1), (5,1), (7,3)] ==> [(2,1), (5,1)]
pgcd [(2,3), (5,2)] [(7,3)]               ==> []
```

Réponse :

## 4 Membres d'un club sportif (5pts)

Nous considérons dans cet exercice les personnes membres d'un club sportif qui sont classées dans des catégories selon leur âge.

Soit `data Categorie = Enfant | Junior | Senior | Veteran deriving (Show)` un type ayant des valeurs pour chaque tranche d'âge.

Soit `type Info = (String, Categorie)` un type représentant des couples de la forme (`prénom`, `catégorie`).

1. Compléter `data Personne = Personne { ... } deriving (Show)` pour définir le type `Personne` avec les informations suivantes pour chaque personne : prénom, année de naissance et la listes des allergies où chacune est donnée par une chaîne de caractères. Par exemple : **(1,5pts)**

```
pa= Personne "Anais" 2012 ["Lait","Noisette"]
pb= Personne "Benoit" 1995 ["Gluten"]
pc= Personne "Claude" 2001 []
pd= Personne "Diane" 2007 ["Noisette"]
```

Réponse :

2. Définir une fonction qui calcule l'âge en 2024 d'une personne donnée. Par exemple : **(1,5pts)**

`calcAge pc ==> 23`

Réponse :

3. Définir une fonction (`calcInfo :: Personne -> Info`) qui renvoie un couple contenant le prénom et la catégorie d'une personne d'après son âge selon le tableau ci-dessous

| intervalle age  | Categorie |
|-----------------|-----------|
| $]-\infty, 12]$ | Enfant    |
| $]12, 20]$      | Junior    |
| $]20, 30]$      | Senior    |
| $]30, +\infty[$ | Veteran   |

auxiliaires.

Indication : on utilisera l'instruction **where** ou une ou plusieurs fonctions

**(2pts)**

`calcInfo pa ==> ("Anais", Enfant)`

`calcInfo pb ==> ("Benoit", Senior)`

Réponse :